Date: .... No: ....

Problem 1: 2,5
Problem 2: 2,5
Problem 3: 2,5
Problem 4: 2,5
10
9,5

Nguyễn Hàm Anh Dũng
~~A~~ E2SCQCE02 - N
Đề

Problem 1: Singly Linked List –
Given a singly linked lists: head -> [10] -> [25] -> [60] -> [5] -> null
- Task 1: Write a function/algorithm in pseudocode
- Task 2: Draw a diagram or step-by-step pointer trace table showing

Problem 2: Circular linked list – Insert at head
Given a singly circular Linked List:
head -> [10] -> [20] -> [30] -> (points back to head)
Task 1: Write pseudocode for Insert(head (head), new Value)
Task 2: Trace the execution steps (via ~~text~~) when inserting new Value = 5 into the given list

Problem 3: Doubly Linked List – Insert Before
Given a doubly linked list containing integers
- Task 1: Write pseudocode for Insert Before de insert a new node, ~~containing~~
- Task 2: List the exact order of pointer assignments required and briefly justify why this specific sequence avoid memory leaks
- Task 3: Explicitly handle the following two edge cases in code/pseudocode:
  1. target is located at the head node
  2. target does not exists in the list

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

Problem 4: Sorted linked list – In Order Insertion

Given a singly linked

Task 1: Write the function SortedInsert(head, val) to insert

Task 2: Construct a step-by-step table detailing

partners.

Answers:

235 Problem 1

1.16 T1: function Reverse(head)

prev ← Null

curr ← head

next ← NULL

while curr ≠ Null:

next ← curr.next — 1. save the next node

curr.next ← prev — 2. reverse the link

prev ← curr — 3. move prev forward

curr ← next — 4. move curr forward

return prev — 5. prev is the new head

Time is O(n) extra space is O(1)

125 Task 2.

Iteration	next (saved)	Action	prev	curr	list state
Start	—	—	Null	10	10→25→40→15→Null
1	25	10.next=Null	10	25	10→Null→25→40→15→Null
2	40	25.next=10	25	40	25→10→Null, 40→15→Null
3	15	40.next=25	40	15	40→25→10→Null, 15→Null

Date: .... No: ....

4 Null / 15.next = 40 15 Null 15→40→25→10→Null
the loop stops because curr = Null. the function returns
prev = 15, so the result is 15
head → [15] → [40] → [25] → [10] → Null

2,5 Problem 2:

1,75 Task 1:

```
function InsertHead(head, newValue)
    node ← new Node(newValue)
    if head == Null
        node.next ← node
        return node
    tail ← head
    while (tail.next ≠ head)
        tail ← tail.next
    node.next ← head
    tail.next ← node
    return node
```

1,25 Task 2:

1. Create node [5]. the list is not empty.
2. Set tail = 10. Since 10.next = 20 ≠ head, tail = 20. Since 20.next = 10 ≠ head tail = 30
Since 30.next = 10 = head, stop.
3. Set 5.next = head (node 10)
4. Set 30.next = 5
5. Return node 5 as the new head

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

Result head -> [5] -> [10] -> [20] -> [30] -> back to 5)

2.5 Problem 3:

0.75 Task 1:

```
function InsertBefore(head, target, newValue)
    cur ← head
    while (cur ≠ Null and cur.data ≠ target):
        cur ← cur.next
    if cur == Null:
        return head
    node ← new Node(newValue)
    node.prev ← cur.prev
    node.next ← cur
    if (cur.prev ≠ Null)
        cur.prev.next ← node
    else:
        head ← node
    cur.prev ← node
    return head
```

0.75 Task 2:

1. node.prev ← cur.prev
2. node.next ← cur
3. cur.prev.next ← node
4. cur.prev ← node

Step 1 and 2 only change the new node, not

Step 3 must come before Step 4. Step 4 overwrite cur.prev, and after that there's no way to reach the old predecessor, which would leave a broken link or a lost node

Date: .... No: ....

Nguyễn Hoàng Anh Dũng
E2S CQCE01-N

the old predecessor and cur are always reachable until they are linked, so on node is orphaned (no memory leak) and no forward or backward link is ever inconsistent

10 Task 3:

1. target is the head cur.prev = Null, so skip step 3 and set head ← node. the new node becomes the new head with node.prev = Null
2. Target does not exist: the loop ends with cur = Null, so return head unchanged without allocating a node

Call an empty list also falls under case 2)

2 problem

1.25 Task 1

```
Sundram Sorted Insert (head, val)
  node ← new Node(val)
  if(head = Null or val ≤ head.data)
    node.next ← head
    return node
  curr ← head
  while( curr.next ≠ Null and curr.next.data < val)
      cur ← cur.next
  node.next ← cur.next
```

Thành Đạt

Scanned with CamScanner

Date: .... No: ....

curr.next ← node
return head

1.5 Task 2:

Step	curr	curr.next	Condition curr.next ≠ Null and curr.next.data<6	Action
Init	2	5	head check: head=Null? No. 6<2? No.	Go to the loop
1	2	5	5≠Null and 5<6→true	curr=5
2	5	8	8≠Null and 8<6→false	Exit loop
Insert	5	8	—	6.next=8, then 5.next=6

Result: head → [2] → [5] → [6] → [8] → Null,